Studio Nx

Backend(FastAPI/Python) 26 — create_access_token() · _create_token()

2026-10-12

Backend(FastAPI/Python) 26 — `create_access_token()` · `_create_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-05 로그인: Access Token 발급 (PRD 01 FR-103, FR-105)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	25 입력 스키마 (P1-04 끝) · 27 `create_refresh_token()`

1. 하는 일

회원 번호로 **30분짜리 Access Token(JWT)**을 만듭니다. 05번 `decode_access_token()`이 읽는 토큰을 만드는 쪽입니다.

`create_access_token(7)` → `"eyJhbGciOiJIUzI1NiIs….eyJzdWIiOiI3Iiwi….kQ3x…"`

토큰 안의 칸	값	뜻
`sub`	`"7"`	토큰의 주인입니다 (회원 번호, 문자열)
`type`	`"access"`	토큰의 종류로, 리프레시 토큰(27)과 구분합니다
`iat`	`1791339232`	만든 시각입니다 (1970년부터 센 초)
`exp`	`1791341032`	만료 시각으로, `iat` + 1800초(30분)입니다

함수	역할
`create_access_token()`	바깥에서 부르는 이름입니다. 수명 30분을 정합니다
`_create_token()`	실제로 토큰을 만드는 곳입니다. 27번 `create_refresh_token()`도 같은 함수를 씁니다 (05번의 `_decode_token`과 짝을 이룹니다)

2. 언제 쓰나

쓰는 곳	화면	하는 일
P1-05 `POST /auth/login` (32)	home.html 로그인 모달	이메일과 비밀번호가 맞으면 응답 `{"access_token": …}`으로 줍니다
P1-05 `POST /auth/refresh` (33)	모든 페이지 (`api.js` 01)	리프레시 쿠키가 유효하면 새 Access Token을 줍니다

받은 토큰은 브라우저 메모리에 두고(`setAccessToken`), 모든 API 요청의 `Authorization: Bearer …`에 붙입니다 (JS 02). 30분이 지나면 서버(05)가 401로 응답하고, 브라우저는 33번으로 새 토큰을 받습니다.

3. 코드

import (파일 맨 위)

```
import re
import string
from datetime import UTC, datetime, timedelta

import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다. **`from datetime import UTC, datetime, timedelta`를 이번에 추가했습니다.** 이 함수들은 이것과 `jwt`, `get_settings`를 사용합니다.

import	쓰는 곳
`UTC`, `datetime`, `timedelta`	지금 시각(UTC)과 30분이라는 기간
`jwt`	`jwt.encode`로 토큰을 만듭니다 (05번의 `jwt.decode`와 반대 방향)
`get_settings`	`.env`의 `JWT_SECRET`과 `jwt_access_minutes`(30)

함수

```
JWT_ALGORITHM = "HS256"


def create_access_token(user_id: int) -> str:
    minutes = get_settings().jwt_access_minutes
    return _create_token(user_id, "access", timedelta(minutes=minutes))


def _create_token(user_id: int, token_type: str, lifetime: timedelta) -> str:
    now = datetime.now(UTC)
    payload = {
        "sub": str(user_id),
        "type": token_type,
        "iat": now,
        "exp": now + lifetime,
    }
    return jwt.encode(payload, get_settings().jwt_secret, algorithm=JWT_ALGORITHM)
```

문법 풀이

import 문법	뜻
`from datetime import UTC, datetime, timedelta`	파이썬 기본 `datetime` 모듈에서 세 이름을 가져옵니다

식	뜻
`timedelta(minutes=30)`	30분이라는 **기간**입니다
`datetime.now(UTC)`	지금 시각입니다 (UTC, 시간대 표시가 있습니다)
`now + lifetime`	30분 뒤의 시각입니다
`jwt.encode(payload, 비밀값, algorithm="HS256")`	머리, 내용, 서명 세 부분을 만들어 `.`으로 이은 문자열입니다. `datetime` 값은 PyJWT가 초 단위 숫자로 바꿔 넣습니다
`str(user_id)`	`7`을 `"7"`로 바꿉니다. JWT 규격에서 `sub`는 문자열이기 때문입니다

식	뜻
`_create_token`의 `_`	이 파일 안에서만 쓰는 함수라는 표시입니다 (05번의 `_decode_token`과 같은 관례)

4. 흐름

1. 설정에서 수명(30분)을 읽습니다.
2. 지금 시각(UTC)을 잽니다.
3. `sub`, `type`, `iat`, `exp`를 담습니다.
4. 서버의 비밀값으로 서명해서 토큰 문자열을 돌려줍니다.

5. 왜 이렇게

선택	이유
수명 30분	이 토큰은 가진 사람을 인정하는 방식(Bearer)이라서, 누가 훔쳐 가더라도 오래 쓰지 못하게 짧게 둡니다. 대신 리프레시(27, 33)를 이용해서 사용자는 계속 로그인된 것처럼 쓸 수 있습니다
수명은 `.env` 설정에서 읽음	코드를 고치지 않고도 바꿀 수 있습니다 (`jwt_access_minutes`)
`type`을 넣음	05번이 `type`을 검사해서, 3일짜리 리프레시 토큰을 Access 토큰 자리에 쓰지 못하게 막습니다
`iat`(만든 시각)를 추가함 (템플릿에는 없음)	같은 회원이 같은 초에 두 번 받지 않는 한 토큰이 매번 달라집니다. 또 나중에 "이 시각 이전의 토큰은 무효" 같은 처리를 할 수 있습니다. 05번은 이 칸을 요구하지 않으므로 영향이 없습니다
만드는 일을 `_create_token`으로 분리	27번 `create_refresh_token`은 수명과 종류만 다르므로 한 줄로 끝납니다
알고리즘은 05번과 같은 `JWT_ALGORITHM`을 씀	만드는 쪽과 읽는 쪽이 같은 상수를 씁니다

토큰 내용은 암호화되지 않습니다 (05 §2). 누구나 읽을 수 있으므로, 회원 번호 외에 이메일이나 권한 같은 정보는 넣지 않습니다.

6. 시험 (`tests/test_security.py`에 5개를 추가해서 모두 14개)

시험	확인한 것
만들고 읽기	`decode_access_token(create_access_token(7)) == 7`입니다. 05번과 짝이 맞습니다
내용	`sub`는 `"7"`, `type`은 `"access"`, `exp - iat`는 1800초입니다
머리	`alg`는 `HS256`입니다
시각	`iat`가 지금 시각과 2초 안으로 차이 나고, `exp`는 `iat + 1800`입니다
회원마다 다름	7번 회원과 8번 회원의 토큰이 다릅니다

05번 시험에서 쓰던 `jwt.encode` 직접 만들기(`make_token`)는 그대로 둡니다. 만료되었거나 위조된 이상한 토큰은 이 함수로 만들 수 없기 때문입니다.

전체 시험 191개가 통과했고, `ruff` 검사도 통과했습니다.